ABRIDGEKV: COMPRESSING LONG REASONING INTO THE KV CACHE OF LEARNED PAUSE TOKENS

Ziyang Jack Cai[1*] **Lijie Yang**[1] **Vasilis Kontonis**[2] **Dimitris Papailiopoulos**[2] **Shivam Garg**[2]
[1]Princeton University [2]Microsoft Research

ABSTRACT

Long chain-of-thought decoding is memory-bound: the KV cache of the generated tokens grows linearly with the test-time compute being spent. Evicting cache entries does not fix this, because a model never trained on a cache with entries removed generates longer, often up to the length limit. We instead train the model to compress its own output. ABRIDGEKV compresses each completed block of generated text into the KV entries of a few learned pause tokens. These are extra input tokens with trained embeddings that produce no output. Once the pause tokens have encoded the block, ABRIDGEKV frees the block's original cache; subsequent tokens attend to the pause tokens' KV entries in its place. Three novel choices make this work: the pause tokens' KV entries take positions inside the block they replace, the cache budget is allocated across layers according to the measured value of each layer's entries, and compression operates on relatively large blocks. On 446 competition problems from AIME, HMMT, AMC, BRUMO, CMIMC and SMT with 32K-token generations, keeping a quarter of the generated cache matches an identically trained full-KV model in accuracy and output length; the strongest published evictor at a matched budget writes far longer, and the closest trained compressor, retrained on the same data, loses 10 points. Served in vLLM, the cache is $3.5\times$ smaller per sequence and a saturated H100 finishes a problem $3.2\times$ faster than with full KV.

1 INTRODUCTION

Test-time scaling improves reasoning accuracy by generating longer chains of thought (CoT) (OpenAI, 2024; DeepSeek-AI, 2025a; Muennighoff et al., 2025; Snell et al., 2024). On competition mathematics, even small models generate tens of thousands of tokens per response and frequently reach the maximum generation length. Decoding such long CoTs is memory-bound: the KV cache grows linearly with the number of generated tokens, and its size, not the model weights, bounds the batch size and hence throughput (Kwon et al., 2023). Unlike long-prompt settings, this cache is produced by the model's own decoding and grows with exactly the test-time compute being spent. We target the KV cache of the generated tokens.

KV eviction keeps a fixed number of cache entries and discards the rest, selected by recency or accumulated attention score (Zhang et al., 2023; Xiao et al., 2024b; Li et al., 2024); several recent methods specialize it to long reasoning (Cai et al., 2025; Song et al., 2025; Zhang et al., 2026; Chang et al., 2026). Two properties limit it. The model is never trained on a cache with entries removed, and the selection is made from past attention, whereas a CoT reuses intermediate results at unpredictable later points; an evicted entry leaves no representation behind. Empirically the failure appears as longer generations. When they keep a quarter of the cache, simple eviction policies (recency, random, accumulated attention) let 38–58% of samples run to the 32K limit, and R-KV, the strongest published method we evaluate, generates 40–69% more tokens than the uncompressed model at the two budgets we test (Figure 4a), which offsets the per-token memory saving. What to keep from a block of reasoning is better decided by a model trained for it: the compression should be learned, and the model should be trained to continue from the compressed history.

*Correspondence to zc5794@princeton.edu.

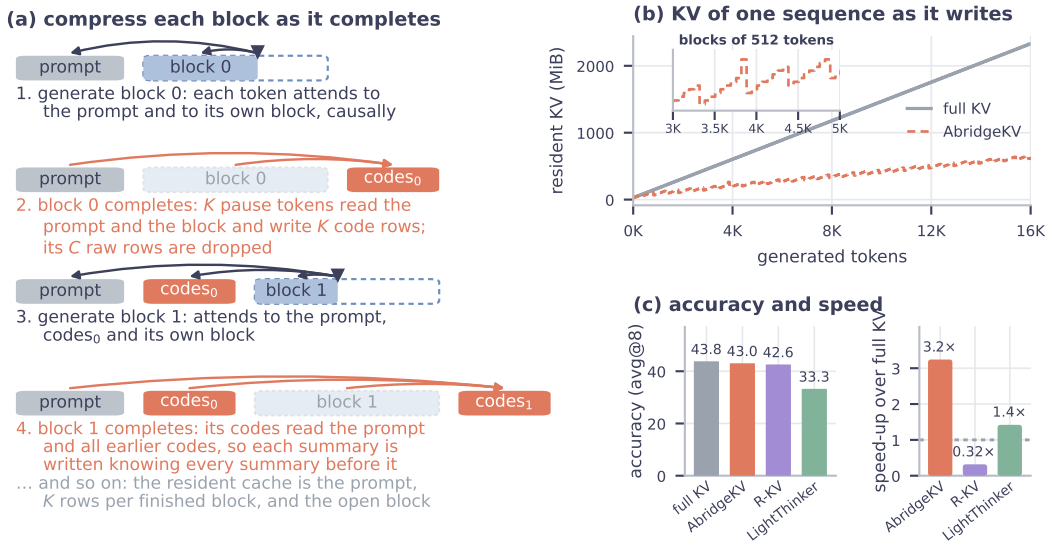


Figure 1: (a) ABRIDGEKV writes in blocks of C tokens; when a block completes, K pause tokens write its code rows (their KV entries, one per pause token per layer) and its raw rows are freed. (b) KV of one sequence in vLLM: it rises through each block and falls at each compression. (c) Accuracy (avg@8) on 446 competition problems at the 32K cap, and speed-up in time to finish a problem relative to full KV (dotted line) on a saturated H100.

Learned compression (Mu et al., 2023; Ge et al., 2024b; Zhang et al., 2024) was developed for fixed inputs, such as long documents, compressed once before decoding. Compressing a CoT differs: the model must compress its own output online, dozens of times per response, and condition on the compressed states thousands of tokens later, so each summary has to be written without knowing which of its contents the later reasoning will need.

MEMENTO (Kontonis et al., 2026), which motivated this work, belongs to a line of work that trains models to compress their own reasoning in text, including InftyThink (Yan et al., 2025), InftyThink+ (Yan et al., 2026), Accordion-Thinking (Yang et al., 2026) and The Markovian Thinker (Aghajohari et al., 2025). MEMENTO splits the model's reasoning into blocks, writes a short summary of each block, and continues from the summaries alone. Producing its training data, however, requires a sophisticated annotation pipeline in which a frontier LLM scores candidate block boundaries, writes the summaries, and grades and refines them. Can we avoid such a pipeline entirely and instead use a fixed set of tokens as summaries, allowing their KV entries to learn from the next-token loss what information to preserve?

LightThinker (Zhang et al., 2025) and Breadcrumbs Reasoning (Monea et al., 2025) pursue this basic idea: rather than producing explicit textual summaries, they replace generated text with the KV entries of learned memory tokens. Their evaluations, however, involve shorter generations and less demanding long-reasoning settings. LightThinker is evaluated with outputs of at most 10K tokens, while Breadcrumbs is evaluated on synthetic tasks with outputs of at most 1,000 tokens. When we retrain LightThinker on our data, serve it in our engine, and evaluate it on long competition-math problems, its accuracy is 10 points below the full-KV model. This drop is perhaps unsurprising: a small, fixed set of KV entries must replace the information carried by an explicit textual summary. More surprisingly, we show that three design choices can recover this loss and make pause-token compression effective at long generation lengths.

Our method, ABRIDGEKV, compresses the generated KV cache in fixed blocks of 512 tokens (Figure 1a). When a block is complete, a set of pause tokens attends to it and encodes a summary in their own KV entries; we call the KV entry of one pause token at one layer a *code row*. The block's original KV entries are then freed, and later tokens attend to the code rows in their place. The model is trained end to end with the next-token loss, so later tokens teach the code rows what

information to preserve. Three design choices are central: code rows take positions inside the block they replace, the cache budget is allocated across layers according to the measured value of each layer's entries, and compression operates on relatively large blocks. Together, these choices make pause-token compression effective over long generations.

The resulting reduction is realized as physical memory savings in our paged serving engine. Because each layer has a fixed budget, vLLM can maintain a separate page pool for each budget level: rows that a layer does not retain are never allocated, and pages belonging to a freed block are returned to the pool for other requests.

Our contributions are as follows.

1. We introduce ABRIDGEKV, in which a reasoning model **compresses its own chain of thought into the KV cache of learned pause tokens** as it writes, trained end to end and served as physical memory in a paged engine (Section 2).
2. We identify **three novel design choices**: in-block code positions (no prior learned summary uses them), a layer budget set by measured value (new to learned compression) and large blocks (the nearest prior method compresses every paragraph, about 50 tokens), and show that code rows carry their block's content many blocks later (Sections 4.4–5).
3. At a 32K cap, ABRIDGEKV **matches full KV with a $4\times$ smaller persistent cache and finishes a problem $3.2\times$ faster** in our tuned vLLM, where eviction writes far longer and LightThinker loses 10 points (Section 4).

Future directions and implications. ABRIDGEKV suggests several promising directions beyond the setting studied here. We highlight two that appear particularly compelling.

First, ABRIDGEKV could work alongside the compaction mechanisms already used by long-running coding agents. Modern agents typically compact their context only after it reaches roughly 250K to 1M tokens, rewriting accumulated file contents, tool outputs, and reasoning into a textual summary before generation resumes. ABRIDGEKV operates at a much finer scale: by compressing every completed block, it reduces the rate at which each request consumes KV memory throughout the long interval before the next compaction. At our 4:1 budget, for example, the generated-token KV grows at roughly one quarter of its original rate. The two mechanisms therefore act at different scales and can naturally be combined. A more ambitious direction is to ask whether recursive ABRIDGEKV, compressing earlier code rows into new code rows, could eventually replace the compaction step altogether and keep the cache bounded without rewriting the history as text.

Second, learned KV compression may be especially promising for modalities that are difficult to summarize faithfully in text. Computer-use agents, for example, must reason over long sequences of screenshots and other visual observations whose task-relevant content may not admit a natural textual summary. In such settings, the KV space may provide a more natural compression medium, allowing the model to learn directly which visual information must be retained for future actions. Extending ABRIDGEKV to multimodal and computer-use agents is therefore an exciting direction for future work.

2 METHOD

2.1 BLOCK LAYOUT AND ATTENTION MASK

Layout. The prompt (P tokens) is kept raw and never compressed. The response is generated in blocks of $C{=}512$ tokens; block $b \in \{0, 1, \dots\}$ has content $x_{b,0}, \dots, x_{b,C-1}$ and $K_{\max}{=}256$ *code slots* $p_{b,0}, \dots, p_{b,K_{\max}-1}$, pause tokens with learned input embeddings. When block b completes, the $K_{\max}$ pause tokens are run through the model in one forward pass; their KV rows at every layer are the block's *codes*. The block's content KV is then dropped. Nothing else is added to the model: no adapters, no extra layers, no change to the rotary embedding.

Visibility. A content query $x_{b,t}$ attends to the prompt, to the codes of every earlier block $b' < b$, and to its own block causally, $x_{b,<t}$. A code query $p_{b,k}$ attends to the prompt, to the codes of earlier blocks, to all of its block $x_{b,0..C-1}$, and to the other codes of its block. Content of block b is never

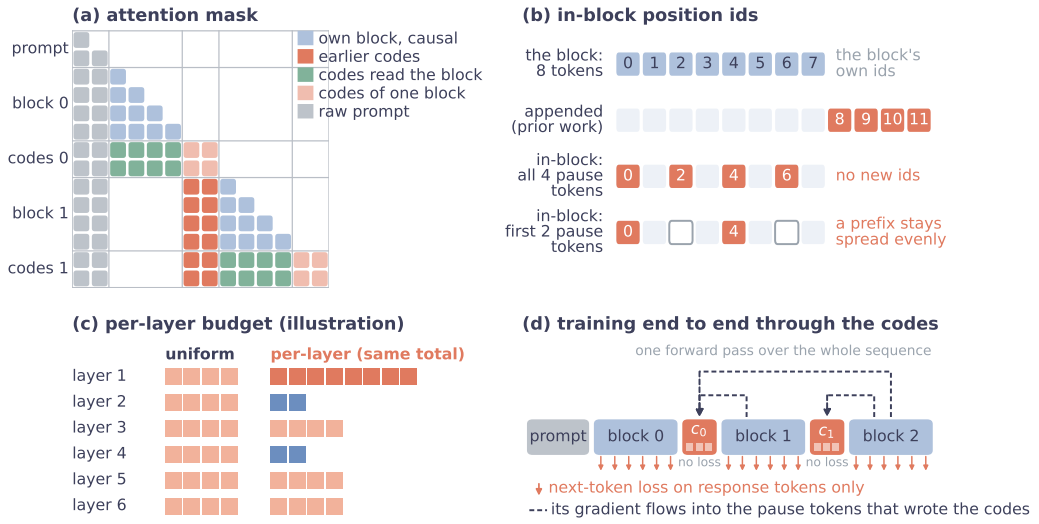


Figure 2: Method details. (a) Attention mask for two blocks (C=4, K=2, 2-token prompt). (b) Pause token k (whose KV entries are code row k at every layer) takes position $\mathrm{bitrev}(k) \cdot C/K_{\max}$ inside the block it replaces (C=8, $K_{\max}$=4), not after it as in prior work; any prefix of the pause tokens is an even grid. (c) Per-layer quotas keep the uniform mean. (d) Training is one forward pass under mask (a), with next-token loss on response tokens only; later blocks' loss reaches the pause tokens that wrote their codes.

visible to any query outside block b (Figure 2a). This mask is what the trainer enforces on the full sequence and what the runtime enforces by eviction.

Positions. Content keeps its uncompressed numbering, and each code row sits *inside* the block it summarizes. The simplest such layout spaces the $K_{\max}$ pause tokens evenly over the block, pause token k at offset $k\,C/K_{\max}$. Even spacing alone already keeps every position within the response length and keeps the distances between later queries and the codes close to those of the original block (properties (i) and (iii) below). The per-layer budgets of Section 2.3 add one requirement: a layer that keeps only its first q pause tokens should still see codes spread over the whole block, not over its first $q\,C/K_{\max}$ tokens. Bit-reversing the order in which the evenly spaced positions are assigned achieves this:

$$\mathrm{pos}(x_{b,t}) = P + bC + t, \qquad \mathrm{pos}(p_{b,k}) = P + bC + \mathrm{bitrev}_{K_{\max}}(k)\,\frac{C}{K_{\max}}, \tag{1}$$

where $\mathrm{bitrev}_{K_{\max}}(k)$ reverses the $\log_2 K_{\max}$ bits of k (for $K_{\max}$=8 the slot order is $0, 4, 2, 6, 1, 5, 3, 7$), so the first q pause tokens always sit on an even grid of stride C/q. Query and key use the same rotation, so this is a choice of position ids only. Three properties follow. *(i) No position inflation.* A response of T tokens occupies exactly T positions whatever $K_{\max}$ is, so a 32K-token response stays inside the 32,768 positions the base model was trained on. Giving the codes fresh positions after their block, as the learned-summary literature does, stretches the same response over $T(C{+}K)/C$ positions. *(ii) Nested, K-independent quotas.* Keeping the first q slots in bit-reversed order, $k < q$, keeps an even grid of stride C/q over the block (Figure 2b): every quota level sees an evenly spaced summary, quota q is a prefix of quota $2q$, and the layout of the kept rows does not depend on $K_{\max}$, so a per-(layer, head) budget is a mask on one trained model rather than a different model. *(iii) Stable relative geometry.* A later query sees each code at roughly the distance the content it replaces would have had, rather than at a distance that grows with the number of intervening summaries. Ablating only the in-block placement (giving the code rows new positions after their block), with the mask, budget, trainer and data fixed, lowers accuracy by 6.2 points of avg@8 (95% interval $[4.6, 7.8]$) at the 32K cap; the loss is not caused by positions beyond the base model's trained range (Section 4.4). The in-block layout has one cost. A pause token's query sits at its in-block position, so the tokens of its block that come after that position are, for

this query, at negative relative offsets: keys positioned after the query. The pretrained weights have never seen such offsets, so fine-tuning has to learn to use them.

Quotas. Each layer ℓ keeps only the first $q_\ell \leq K_{\max}$ code rows of every completed block and masks the rest; at serving time only those rows persist (Appendix E). Section 2.3 sets the quotas.

2.2 Training objective

The model is trained with the ordinary next-token loss on every response token, computed while its history is held as codes (Eq. 2); the prompt is never compressed. The whole response, code rows included, is one forward pass under the block mask, so the loss of later blocks flows back into the pause tokens that wrote the codes they read (Figure 2d). We add no auxiliary terms: distillation from a frozen full-context teacher and keeping a raw tail were tried and did not help (Appendix B.1).

$$\mathcal{L}_{\mathrm{CE}} = -\tfrac{1}{|R|} \sum_{t \in R} \log p_\theta\big(x_t \mid \text{prompt},\ \text{codes}_{<b(t)},\ x_{b(t),<t}\big) \tag{2}$$

2.3 Allocating the budget across layers

The budget is a per-layer quota $q_\ell \leq K_{\max}$: layer ℓ keeps q_ℓ of the $K_{\max}$=256 code rows of every completed block and masks the rest, the same quota for every KV head of the layer (so tensor-parallel serving stays balanced and the physical cache can drop the masked rows, Appendix E). *Uniform* allocation sets q_ℓ=128 everywhere. *Per-layer* (dynamic) allocation keeps the same mean of 128 rows but moves rows between layers, in three steps:

1. Train a model with the uniform-128 schedule.
2. Measure what each layer's code rows are worth: at evaluation time, without retraining, halve one layer's quota (128 $\rightarrow$ 64) and record the increase in held-out cross-entropy; repeat for each of the 36 layers (Figure 6a). Most layers cost nothing; a few cost a lot.
3. Rank layers by cost: the 6 most expensive get 256 rows, the 12 cheapest 64, the other 18 keep 128 ($6{\cdot}256 + 18{\cdot}128 + 12{\cdot}64 = 36{\cdot}128$), and train a new model on this schedule.

The resulting schedule (Figure 6) raises layers 0, 15, 18, 20, 21 and 24 and cuts layers 1–6, 8, 10, 30–32 and 35. Section 5 explains what the measurement shows, why we use three levels and cut only a third of the layers, and why the quota is set per layer rather than per head.

3 Experimental setup

Every arm fine-tunes Qwen3-4B-Base on the same 2B response tokens of math SFT data (400K prompt/response pairs derived from OpenMathReasoning) in the same canonical order, with C=512, $K_{\max}$=256 slots and a mean budget of 128 code rows per block unless stated; each arm is served with the schedule it was trained with. Arms are named by their block-to-code ratio (this budget is 4:1; 2:1 and 8:1 keep 256 and 64 rows) and their allocation, uniform or per-layer; *appended positions* is the ablation whose code rows take new positions after the block. Full hyperparameters, hardware and throughput are in Appendix B.1. The reference is the same base model fine-tuned on the same tokens with full KV, the ceiling a compressed model can match but not beat; every arm is reported as a gap to it.

Evaluation. A 446-problem competition suite (twelve competitions from AIME, HMMT, BRUMO, CMIMC, SMT and AMC, 2022–2026; Appendix B.2), 8 samples per problem at T=0.6, top-p 0.95, top-k 20, seed 1234, and a 32K cap (31,744 generated tokens); a sample is correct if its last boxed answer matches or math-verify judges the full text equivalent. Every arm is scored on the same problems and seed, so differences are paired per problem, with a 2000-resample per-problem bootstrap. Pooling makes these comparisons resolvable: on one 30-problem set the 95% interval on a paired gap is about $\pm$5 points, and a full-KV re-run differing only in its data seed (the same data pool, with a different random draw of which examples each training step uses, and in what order) scores significantly below the original full-KV model on AIME 2024 and above it on AIME 2025; on the pooled suite the half-width is 1.5 (Appendix B.2). We report avg@8, the mean accuracy

Table 1: Main comparison at the 32K cap. Suite: avg@8 over 446 problems with 95% bootstrap intervals; Δ: paired difference to the full-KV model (vanilla). KV/512: persistent KV rows per 512 generated tokens. trunc.: share of suite samples reaching the cap. † Eviction baselines, run on a 60-problem subset (AIME 2025 and HMMT Feb 2025) on which full KV scores 29.8 and the per-layer arm 28.3: their Δ is paired on those problems and comparable with the other rows' Δ, their avg@8 is not. Protocol details: Appendix B.2. Shaded: ours; bold: best below the reference rows.

Arm	KV/512	CE	MATH-500	Suite avg@8	Δ vs. vanilla	trunc.%
References						
Base model (no SFT)	512	–	69.6	13.5 [11.0, 16.2]	-30.4 [-33.4, -27.0]	5.6
Vanilla SFT (full KV)	512	0.4162	88.1	43.8 [39.8, 47.6]	–	3.4
Ours: dynamic per-layer allocation						
ABRIDGEKV (per-layer)	128	**0.4215**	87.5	**43.0** [39.1, 46.9]	**-0.8** [-2.3, +0.8]	**3.2**
Ablations: one design choice removed						
uniform allocation	128	0.4227	**88.3**	41.5 [37.6, 45.3]	-2.3 [-3.8, -0.9]	3.4
appended positions	128	0.4297	85.8	35.3 [31.6, 39.0]	-8.5 [-10.3, -6.8]	3.6
Prior learned compression, in our training and serving stack						
LightThinker (thought-level)	97	0.4391	83.4	33.3 [29.6, 37.0]	-10.5 [-12.5, -8.7]	5.4
Training-free eviction (vanilla weights)						
sink + recency	128	–	84.0	22.7 [14.6, 32.3]†	-7.1 [-10.4, -3.8]†	37.9
random	128	–	76.1	18.1 [11.0, 26.2]†	-11.7 [-16.5, -7.3]†	52.9
attention score (H2O)	128	–	59.1	10.0 [5.0, 15.8]†	-19.8 [-26.2, -13.3]†	57.7
R-KV, budget 2048	74	–	83.9	35.6 [31.7, 39.5]	-8.2 [-10.2, -6.3]	55.8
R-KV, budget 4096	144	–	86.5	42.6 [38.5, 46.6]	-1.2 [-2.7, +0.3]	31.7

over the 8 samples of each problem; MATH-500 is scored as avg@4 at a 16K cap. We measure budget as the number of persistent KV rows kept per 512 generated tokens per KV head ("KV/512" in Table 1).

Serving. Speed is measured in vLLM on one H100 with our patch for compressed decoding. A completed block's code rows are computed in the next engine step alongside other requests' decode tokens and its raw rows are released; a per-layer schedule is stored as one KV cache group per quota level, so rows a layer does not keep occupy no memory. Every arm runs under CUDA graphs and vLLM's asynchronous scheduling, except R-KV, whose port applies each compaction before the next step, and LightThinker (Appendix C.2). Time to finish is measured by replaying 512 requests at each arm's own suite lengths, run to completion at the concurrency cap that finishes fastest. The served model is the trained one: vLLM greedy tokens match the reference decoder, and trainer and incremental generator agree on per-token CE to 4×10^{-4} (Appendix E). Implementation and per-arm measurements are in Appendix E.

4 RESULTS

4.1 ACCURACY AND GENERATION LENGTH

At a quarter of the persistent cache, ABRIDGEKV reaches 43.0 avg@8 against 43.8 for full KV (Table 1, Figure 3a), a paired gap of -0.8 $[-2.3, +0.8]$ that lies inside the spread of a full-KV re-run differing only in its data seed (1 s.d. 0.8 points, Appendix B.1). On MATH-500, whose solutions are short, the two are 87.5 and 88.1.

Generation length. Matching accuracy could hide a model that writes more to make up for what it has lost. ABRIDGEKV does not: its length distribution is statistically indistinguishable from full KV's, as close as a full-KV re-run that differs only in its data seed. The Kolmogorov–Smirnov distance from full KV's length distribution is 0.024 for ABRIDGEKV and 0.017 for the re-run; at our sample size a distance of 0.032 would be significant at the 5% level, so neither is. The same problems also run long under both: the per-problem correlation of generated length with full KV is 0.93 for ABRIDGEKV and 0.92 for the re-run (Figure 4a). R-KV at budget 4096, the budget matched

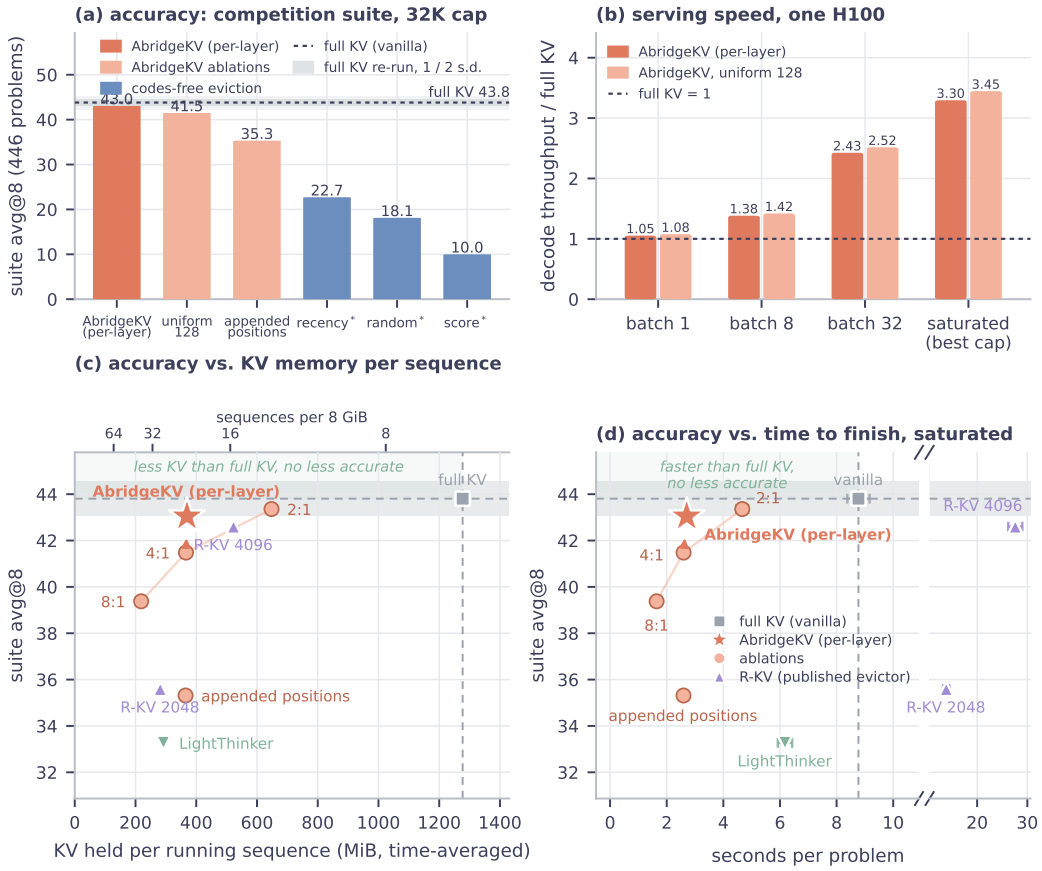


Figure 3: Main results. Orange circles are our arms and the orange star the per-layer model; grey squares full KV, purple triangles R-KV, green LightThinker; grey band: 1 s.d. of a full-KV re-run. (a) Suite avg@8, 32K cap (*60-problem subset). (b) Decode throughput relative to full KV on one H100, for sequences of 16K tokens. (c) Accuracy against KV held per running sequence, time-averaged over each arm's generations; the arrow marks the accuracy gain of the per-layer schedule over the uniform one at the same memory. (d) Accuracy against time to finish a problem: 512 suite-length requests run to completion at the fastest concurrency cap; shaded: at least as accurate as full KV and faster; the axis is broken to show R-KV; its horizontal bar extends to the time R-KV would take if computing its eviction scores took no time. Other horizontal bars: 95% intervals on mean generated length.

to our 4:1 arm, is far from both: the Kolmogorov–Smirnov distance between its length distribution and full KV's is 0.33, and 32% of its samples run to the 32K cap.

4.2 SERVING EFFICIENCY

ABRIDGEKV's resident KV per sequence is $3.54\times$ smaller than full KV's (the ceiling is $4\times$; the prompt and the open block stay raw), and because the per-layer schedule is a handful of cache groups on the stock paged path, it decodes as fast as the uniform setting: $1.05\times$, $1.38\times$ and $2.43\times$ full KV's throughput at batch 1, 8 and 32 (Figure 3b). Averaged over a whole generation, a running ABRIDGEKV sequence holds 369 MiB of KV, against 1,277 MiB for full KV. This is the same memory as the uniform setting, while ABRIDGEKV's accuracy is 1.6 points higher. At these sizes an 8 GiB cache holds 22 ABRIDGEKV sequences instead of 6 full-KV sequences (Figure 3c). Figure 3d then charges each arm for what it generates: saturated, ABRIDGEKV finishes a problem in $0.31\times$ full KV's time, the same as the uniform setting ($0.30\times$). It is the only compressed arm inside the shaded region of Figure 3d, which contains arms at least as accurate as full KV and faster than it.

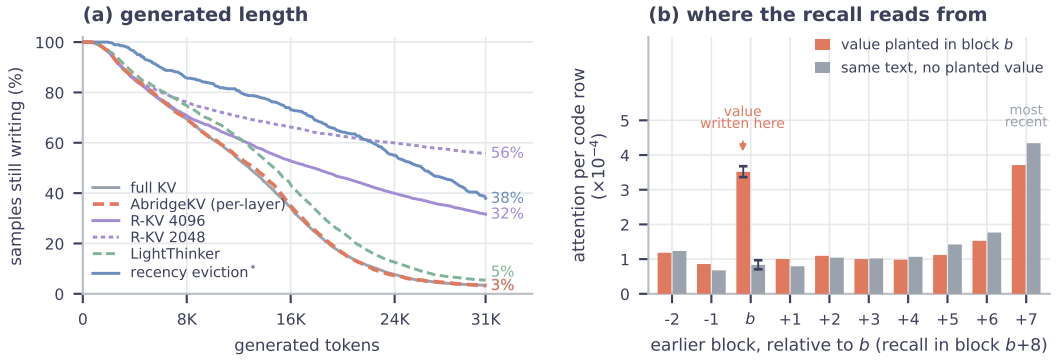


Figure 4: (a) Share of samples still writing at each length, and truncated at the cap (*60-problem subset). (b) A value planted in block b ("$Q_7 = 7391$") is recalled 8 compressions later. The plot shows the attention that the recalled digits pay to each earlier block's code rows, per code row, averaged over heads and layers, with and without the planted value; 93 edited solutions, with a 95% interval at block b.

R-KV (Section 4.3) takes longer than full KV to finish a problem, because it generates more tokens and runs a compaction every 64 decode steps.

4.3 Comparison with eviction

Training-free eviction at the same budget does not come close, which is expected for two reasons. Eviction selects entries by the attention they have received so far, but a CoT reuses intermediate results at later points that past attention does not predict; and an evicted entry leaves no representation, whereas a learned code row retains a summary of every block in a form the model is trained to read. When a needed result has been evicted, the model tends to recompute it, and generations lengthen.

Applied to the full-KV model, eviction that keeps 128 of each completed block's 512 rows, selected by recency, at random, or by accumulated attention score, loses 7–20 points of avg@8 against full KV. These three policies were run only on a 60-problem subset (AIME 2025 and HMMT Feb 2025), and the differences are paired on those problems. The truncation column of Table 1 shows the mechanism: 38–58% of their samples run to the 32K cap. R-KV (Cai et al., 2025), the published evictor designed for long reasoning, which we port to our serving stack and run on the same checkpoint, problems, sampling and grader, is the strongest of the family. We run it in its servable form, one kept set shared by all heads as in its authors' own SGLang port, since a paged cache cannot keep different tokens per head (Appendix C.1). We compare each R-KV budget with the arm whose cache it most nearly matches, giving R-KV the slightly larger cache: a fixed budget of 4096 rows leaves about 12% more rows resident than our 4:1 arm, and 2048 about 12% more than our 8:1 arm (Appendix C.1). At budget 4096, R-KV's accuracy is within noise of AbridgeKV at 4:1 (paired difference -0.4 $[-2.1, +1.2]$ avg@8 points). At budget 2048, it is below our 8:1 arm (-3.8 $[-5.5, -2.2]$). Like the simple eviction policies, R-KV lengthens the output: 32% of its samples reach the cap at budget 4096 and 56% at budget 2048, and it generates 40–69% more tokens than full KV on average (40% at budget 4096, 69% at 2048; Figure 4a), so the memory it saves per token is spent on extra tokens and it takes longer than full KV to finish a problem (Figure 3d). The full comparison, including the port's parity check, is in Appendix C.1.

4.4 Ablations

The ablation rows of Table 1 remove one design choice at a time, holding trainer, data, order, block size and mean budget fixed; Appendix D.3 gives the full block-size and budget sweep.

Code positions. Giving each code row a fresh position after its block, the convention of learned summaries (Zhang et al., 2024; Chevalier et al., 2023; Mu et al., 2023; Zhang et al., 2025), lowers avg@8 by 6.2 $[4.6, 7.8]$ points relative to the uniform arm with in-block positions, the largest effect

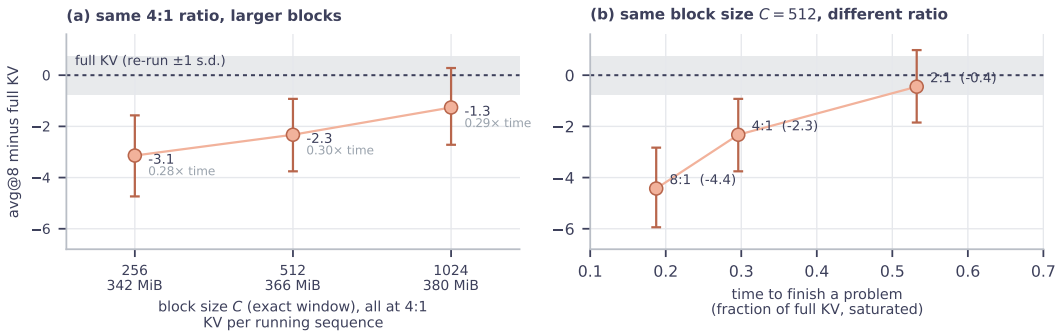


Figure 5: Compression shape; paired gap to full KV with 95% intervals (grey: 1 s.d. of a full-KV re-run). (a) At the same 4:1 ratio, larger blocks hold slightly more KV per sequence (second row of tick labels) and go through fewer lossy compressions; accuracy rises and time to finish (grey) stays the same. (b) At the same block size, a higher ratio is faster and less accurate.

we measure (the -8.5 in Table 1 is its gap to full KV). The loss is not caused by position inflation. With appended positions, a response exceeds the positions the base model was trained on only after $26{,}214$ generated tokens, and 6.3% of samples reach that length. Restricting the comparison to the 256 problems on which no sample of either arm reaches it leaves the gap unchanged at 6.2 $[4.1, 8.3]$. An in-block row keeps the distance to later queries that its content would have had (Appendix A).

Per-layer allocation. Spending the mean of 128 rows uniformly across layers loses 2.3 $[0.9, 3.8]$ points against full KV. Allocating the same mean by each layer's measured value (Section 2.3) recovers 1.6 $[0.1, 3.0]$ of these points, measured as a paired difference between the per-layer and uniform arms, and leaves a gap to full KV (-0.8 $[-2.3, +0.8]$) whose interval contains zero. The difference is positive on every subset of the suite we examined, but no subset on its own is large enough to resolve a difference of this size (Appendix B.2).

Block size at a fixed ratio. A larger block keeps a longer window of recent tokens exact, which costs memory, and lowers the number of lossy compressions the history passes through: at 4:1, a query at our mean generation length comes after 51 compressed blocks at $C{=}256$ and 13 at $C{=}1024$. Suite accuracy increases with block size: 40.7, 41.5 and 42.5 avg@8 for $C{=}256$, 512 and 1024 (Figure 5a). Over this range the KV held per running sequence grows by 11% (342 to 380 MiB), while time to finish does not change measurably (0.28, 0.30 and $0.29\times$ full KV's). Only the smallest block is significantly below full KV: the gap is -3.1 $[-4.7, -1.6]$ at $C{=}256$ and -1.3 $[-2.7, +0.3]$ at $C{=}1024$. Thought-level compression sits far to the small side of this range. LightThinker's algorithm, trained on our data and served in our engine, ends a thought at each paragraph break, every 53 tokens on average in its generations, and compresses the thought into 10 rows (9 cache tokens and its continue token), a ratio of about 5.3:1. It scores 33.3 on the suite, -10.5 $[-12.5, -8.7]$ against full KV (Table 1; Appendix C.2); our 8:1 arm, which keeps fewer rows, loses 4.4. It is also slower: it takes $0.70\times$ full KV's time to finish a problem, against $0.31\times$ for AbridgeKV (Figure 3d), since its compression step runs ten times as often. Its measured time is an upper bound, because it ran with synchronous scheduling (Appendix C.2). Training cost does not go up with block size over this range (Appendix D.3).

Compression ratio at a fixed block size. At $C{=}512$, the 2:1 arm is statistically indistinguishable from full KV (-0.4 $[-1.8, +1.0]$) and finishes a problem in $0.53\times$ full KV's time. With uniform allocation, the 4:1 arm loses 2.3 points and takes $0.30\times$ full KV's time, and the 8:1 arm loses 4.4 points and takes $0.19\times$ (Figure 5b). The per-layer schedule brings the 4:1 arm to parity with full KV at the same time to finish.

5 Analysis

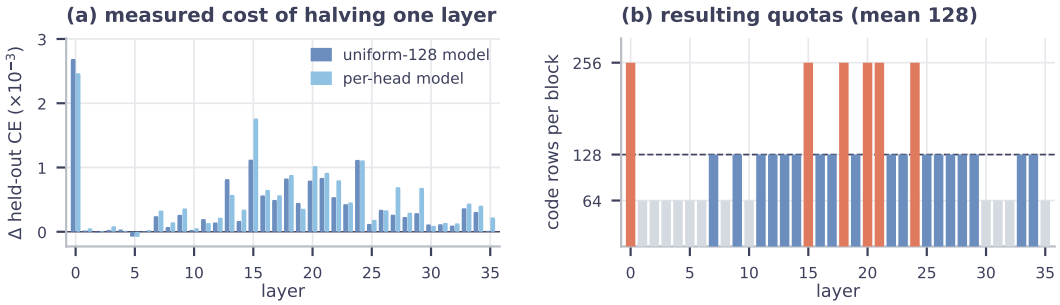


Figure 6: From measurement to quotas. (a) Held-out CE cost of halving one layer's code rows at evaluation time, on two trained models. (b) The trained schedule: the six most expensive layers get 256 rows, the twelve cheapest 64, at the same mean.

5.1 Choosing the quotas

Question. Allocating rows by layer turns a significant loss into parity (Section 4.4). We want to know whether the gain comes from *where* the rows go, which would make the measurement behind the schedule essential, or merely from making the budget uneven.

Procedure. On the trained uniform model we halve one layer's code rows at evaluation time and record the increase in held-out cross-entropy, for each of the 36 layers (Figure 6a). The most expensive layers are raised, the cheapest cut, at the same mean, and the model is retrained on that schedule (Figure 6b). For contrast we train a schedule of the same mean chosen without the trained model, from the heads whose full-KV attention is hardest to approximate with few keys; it raises the early layers.

Result. The measured cost sits in a few layers, the first layer and a band in the middle of the network, while most early and late layers can lose half their rows at almost no cost. Moving rows by this measurement reaches parity with full KV; the schedule chosen from the full-KV model's attention, which raises exactly the early layers the measurement finds cheap, is no better than uniform (Appendix D.1, Table 7). The placement is what matters, and it has to be read from the trained compressed model: a proxy from the uncompressed model points the wrong way. Two refinements did not improve on this schedule. First, the single-layer costs are not additive: halving every layer at once raises held-out cross-entropy by twice the sum of the single-layer increases, so rather than fitting small differences between layers, we cut only a third of the layers and use three quota levels. Second, a per-head schedule matches the per-layer accuracy but costs memory or speed to serve (Appendix D.1).

5.2 Information flow through the code rows

Question. Once a block is compressed its raw KV is gone, so anything later reasoning takes from that block must pass through its code rows. We want to see this directly: whether a specific fact written in a block is stored in that block's code rows and retrieved from them many blocks later.

Procedure. We insert one sentence defining a random four-digit value ("Denote this intermediate result as $Q_7 = 7391$.") into block b of the per-layer model's own AIME solutions, ask for it k blocks later ("Recall that $Q_7 =$"), and record whether it is reproduced and where its tokens attend.

Result. Recall is scored by teacher forcing: after the cue, the value counts as recalled if the model's most likely next tokens are exactly its four digits, so each case is a deterministic yes or no rather than a sample. Over 93 edited solutions from 51 AIME problems, each asked at 1, 2, 4 and 8 compressions after block b (372 cases), the value is recalled in 73% of cases (95% interval 66–79%), against 98% for the full-KV model on the same edited sequences. The rate does not decline from one to eight compressions, and it is zero when block b's code rows are removed. The recalling tokens attend to block b's code rows far more than to any other block's and far more than the same

positions do when no value was planted (Figure 4b): the information travels by the most direct route, from the block into its own code rows and from there to the token that needs it. This routing happens in the 256-row layers, the ones the measurement of Section 5.1 raised, and in the 128-row layers. Failed recalls attend to the right rows as strongly as successful ones, so what limits recall is how much the code rows hold, not whether the model finds them. The same holds without a planted value: in the model's own solutions, when the next block reuses a number first written in block b, the probability the model assigns to the number's later digits is 0.99 with block b's code rows intact and about 0.69 when they are replaced by another block's rows or removed (Figure 8, Appendix D.2).

6 DISCUSSION

Scope. Mathematical reasoning is the common test bed of the closest related work (Kontonis et al., 2026; Zhang et al., 2025; Cai et al., 2025), and it is ours. It is a demanding test of long generation: solutions run to tens of thousands of tokens, correctness is verifiable, and our suite pools twelve competitions over five years, 446 problems, so the claims above are resolved at a precision single benchmarks do not reach. Nothing in the recipe is specific to the domain or the model: the trainer, the layout and the allocation procedure take any decoder with rotary positions, the allocation is re-measured on each trained model rather than transferred, and the serving path is the stock paged engine with one cache group per quota. A first coding-domain comparison at prototyping scale (Appendix D.4) shows the same ordering of full KV, per-layer and uniform, and a per-layer schedule measured on code that coincides with the one measured on mathematics; larger models may also be more compressible, since their KV states carry more redundancy per token.

Block size. Accuracy is still rising at $C{=}1024$, so larger blocks may be better still; we did not train them, although they cost little more to train (Appendix D.3). The trade-off is memory: a larger block keeps more raw rows resident per sequence, so at a fixed cache it means fewer sequences in flight. We expect the practical choice to be the largest block the deployment's cache can carry.

Readable reasoning and other modalities. Reasoning in a latent space instead of text (Hao et al., 2024) shortens generations but gives up the readable chain of thought that makes a model's reasoning possible to monitor (Korbak et al., 2025). ABRIDGEKV keeps the reasoning in text: every token the model writes is ordinary text that can be read and checked, and only the cache that later tokens attend to is compressed. Because the summaries are learned from the next-token loss rather than written out as text, the same recipe also applies where a faithful text summary is hard to define, such as long contexts of images or other non-text inputs, whose KV could be compressed in the same way.

Limitations. Every result starts from a base model fine-tuned with our own SFT. This matches how reasoning models are commonly built, supervised fine-tuning on long reasoning traces followed by reinforcement learning (Abdin et al., 2025), and compression training fits into that SFT stage. It fits equally into the distillation step of pipelines that train separate domain-specific specialists with RL and then distill them into one model (DeepSeek-AI, 2025b). We did not train it directly on a released RL-tuned checkpoint, and whether further fine-tuning there preserves the gains of RL is untested. The compression ratio is fixed per block, so a block that carries little and one that carries a lot get the same rows. And the quota schedule comes from single-layer measurements that we assumed to be independent (Section 5.1); a method that measured how cuts interact could find a better one.

REPRODUCIBILITY STATEMENT

Every arm is trained from Qwen3-4B-Base on the same 2B response tokens in the same order (the canonical data stream), with the hyperparameters, hardware and throughput of Appendix B.1; the 446-problem suite, its sources and the sampling protocol are in Section 3 and Appendix B.2; the serving implementation, the benchmark protocol (pinned CPU cores, CUDA graphs on every arm, the shared concurrency-cap grid) and the per-arm measurements are in Appendix E; and the parity checks that tie the served model to the trained one are in Appendix E. We will release the training code, the vLLM patch, the evaluation and benchmark scripts, and the trained checkpoints.

AI USE STATEMENT

In this work, we used generative AI tools (a coding agent) for implementing methods, including the training, evaluation, serving and benchmark code; for running and monitoring experiments; for drafting and editing text; for creating figures and tables from the logged results; and for searching and summarizing the literature. We have not used generative AI tools for research ideas or for interpreting results, and we have not used them for generating synthetic datasets: our SFT data are derived from OpenMathReasoning and OpenCodeReasoning, public datasets of model-written solutions that we did not generate. Developing theoretical models, formulating mathematical claims, providing or writing proofs, translation, and qualitative data analysis are not applicable to this work. Every reported number was produced by code we ran and was checked against the underlying logs. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

REFERENCES

Marah Abdin, Sahaj Agarwal, Ahmed Awadallah, Vidhisha Balachandran, Harkirat Behl, Lingjiao Chen, et al. Phi-4-reasoning technical report. *arXiv preprint arXiv:2504.21318*, 2025.

Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J. Nair, Ilya Soloveychik, and Purushotham Kamath. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Milad Aghajohari, Kamran Chitsaz, Amirhossein Kazemnejad, Sarath Chandar, Alessandro Sordoni, Aaron Courville, and Siva Reddy. The Markovian thinker: Architecture-agnostic linear scaling of reasoning. *arXiv preprint arXiv:2510.06557*, 2025.

Wasi Uddin Ahmad, Sean Narenthiran, Somshubra Majumdar, Aleksander Ficek, Siddhartha Jain, Jocelyn Huang, Vahid Noroozi, and Boris Ginsburg. OpenCodeReasoning: Advancing data distillation for competitive coding. *arXiv preprint arXiv:2504.01943*, 2025.

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023. EMNLP 2023.

William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. Reducing transformer key-value cache size with cross-layer attention. *arXiv preprint arXiv:2405.12981*, 2024. NeurIPS 2024.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

Zefan Cai, Wen Xiao, Hanshi Sun, Cheng Luo, Yikai Zhang, Ke Wan, Yucheng Li, Yeyang Zhou, Li-Wen Chang, Jiuxiang Gu, Zhen Dong, Anima Anandkumar, Abedelkadir Asi, and Junjie Hu. R-kv: Redundancy-aware kv cache compression for reasoning models. In *Advances in Neural Information Processing Systems*, 2025.

Ting-Yun Chang, Harvey Yiyun Fu, Deqing Fu, Chenghao Yang, Jesse Thomason, and Robin Jia. Value-aware stochastic kv cache eviction for reasoning models. *arXiv preprint arXiv:2606.03928*, 2026.

Jeffrey Cheng and Benjamin Van Durme. Compressed chain of thought: Efficient reasoning through dense representations. *arXiv preprint arXiv:2412.13171*, 2024.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of EMNLP*, pp. 3829–3846, 2023.

DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

DeepSeek-AI. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025a. doi: 10.1038/s41586-025-09422-z.

DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025b.

Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. In *Advances in Neural Information Processing Systems*, 2025.

Yu Fu, Zefan Cai, Abedelkadir Asi, Wayne Xiong, Yue Dong, and Wen Xiao. Not all heads matter: A head-level kv cache compression method with integrated retrieval and reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.

Yizhao Gao, Shuming Guo, Shijie Cao, Yuqing Xia, Yu Cheng, Lei Wang, Lingxiao Ma, Yutao Sun, Tianzhu Ye, Li Dong, Hayden Kwok-Hay So, Yu Hua, Ting Cao, Fan Yang, and Mao Yang. Seerattention-r: Sparse attention adaptation for long reasoning. *arXiv preprint arXiv:2506.08889*, 2025.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. In *International Conference on Learning Representations (ICLR)*, 2024a.

Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations (ICLR)*, 2024b.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.

Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024. NeurIPS 2024.

Jang-Hyun Kim, Jinuk Kim, Sangwoo Kwon, Jae W. Lee, Sangdoo Yun, and Hyun Oh Song. Kvzip: Query-agnostic kv cache compression with context reconstruction. In *Advances in Neural Information Processing Systems*, 2025.

Vasilis Kontonis, Yuchen Zeng, Shivam Garg, Lingjiao Chen, Hao Tang, Ziyan Wang, Ahmed Awadallah, Eric Horvitz, John Langford, and Dimitris Papailiopoulos. MEMENTO: Teaching LLMs to manage their own context. *arXiv preprint arXiv:2604.09852*, 2026.

Tomek Korbak, Mikita Balesni, Elizabeth Barnes, Yoshua Bengio, Joe Benton, Joseph Bloom, et al. Chain of thought monitorability: A new and fragile opportunity for AI safety. *arXiv preprint arXiv:2507.11473*, 2025.

Anna Kuzina, Maciej Pioro, Paul N. Whatmough, and Babak Ehteshami Bejnordi. Kava: Latent reasoning via compressed kv-cache distillation. In *International Conference on Learning Representations (ICLR)*, 2026.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024.

Xin Liu, Runsong Zhao, Xinyu Liu, Junhao Ruan, Pengcheng Huang, Shichao Dong, Chunyang Xiao, Chenglong Wang, Changliang Li, Jingbo Zhu, and Tong Xiao. Hybridthinker: Efficient chain-of-thought reasoning via compressed memory and transient thought steps. *arXiv preprint arXiv:2606.03768*, 2026.

Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36, 2023.

Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024. ICML 2024.

Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. In *Advances in Neural Information Processing Systems*, 2023.

Giovanni Monea, Yair Feldman, Shankar Padmanabhan, Kianté Brantley, and Yoav Artzi. Breadcrumbs reasoning: Memory-efficient reasoning with compression beacons. *arXiv preprint arXiv:2510.13797*, 2025.

Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, 2023.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. In *Proceedings of EMNLP*, 2025.

Piotr Nawrot, Adrian Łańcucki, Marcin Chochowski, David Tarjan, and Edoardo M. Ponti. Dynamic memory compression: Retrofitting llms for accelerated inference. *arXiv preprint arXiv:2403.09636*, 2024. ICML 2024.

OpenAI. Learning to reason with LLMs. `https://openai.com/index/learning-to-reason-with-llms/`, 2024.

Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. Transformers are multi-state rnns. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 18724–18741, 2024.

Jianhui Pang, Fanghua Ye, Derek F. Wong, Xin He, Wanshun Chen, and Longyue Wang. Anchor-based large language models. *arXiv preprint arXiv:2402.07616*, 2024. Findings of ACL 2024.

Guanghui Qin, Corby Rosset, Ethan C. Chau, Nikhil Rao, and Benjamin Van Durme. Dodo: Dynamic contextual compression for decoder-only lms. In *Proceedings of ACL*, 2024.

Ziran Qin, Yuchen Cao, Mingbao Lin, Wen Hu, Shixuan Fan, Ke Cheng, Weiyao Lin, and Jianguo Li. Cake: Cascading and adaptive kv cache eviction with layer preferences. In *International Conference on Learning Representations (ICLR)*, 2025.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Jiwon Song, Dongwon Jo, Yulhwa Kim, and Jae-Joon Kim. Reasoning path compression: Compressing generation trajectories for efficient llm reasoning. *arXiv preprint arXiv:2505.13866*, 2025. NeurIPS 2025.

Wenhui Tan, Jiaze Li, Jianzhong Ju, Zhenbo Luo, Ruihua Song, and Jian Luan. Think silently, think fast: Dynamic latent compression of llm reasoning chains. In *Advances in Neural Information Processing Systems*, 2025.

Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Shikuan Hong, Yiwu Yao, and Gongyi Wang. Razorattention: Efficient kv cache compression through retrieval heads. In *International Conference on Learning Representations (ICLR)*, 2025.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. In *International Conference on Machine Learning (ICML)*, 2024.

Zhongwei Wan, Xinjian Wu, Yu Zhang, Yi Xin, Chaofan Tao, Zhihong Zhu, Xin Wang, Siqi Luo, Jing Xiong, Longyue Wang, and Mi Zhang. D2o: Dynamic discriminative operations for efficient long-context inference of large language models. In *International Conference on Learning Representations (ICLR)*, 2025.

Heng Wang, Jielin Qiu, Wenting Zhao, Cheng Qian, Liangwei Yang, Jiawei Han, Heng Ji, Silvio Savarese, Shelby Heinecke, and Huan Wang. Random attention: Rethinking kv cache eviction for efficient reasoning. *arXiv preprint arXiv:2609.03430*, 2026.

Zihao Wang, Bin Cui, and Shaoduo Gan. Squeezeattention: 2d management of kv-cache in llm inference via layer-wise optimal budget. *arXiv preprint arXiv:2404.04793*, 2024.

Heming Xia, Chak Tou Leong, Wenjie Wang, Yongqi Li, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. In *Proceedings of EMNLP*, 2025.

Chaojun Xiao, Pengle Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. Infllm: Training-free long-context extrapolation for llms with an efficient context memory. In *Advances in Neural Information Processing Systems*, 2024a.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024b.

Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025.

Yuchen Yan, Yongliang Shen, Yang Liu, Jin Jiang, Mengdi Zhang, Jian Shao, and Yueting Zhuang. InftyThink: Breaking the length limits of long-context reasoning in large language models. *arXiv preprint arXiv:2503.06692*, 2025.

Yuchen Yan, Liang Jiang, Jin Jiang, Shuaicheng Li, Zujie Wen, Zhiqiang Zhang, Jun Zhou, Jian Shao, Yueting Zhuang, and Yongliang Shen. InftyThink+: Effective and efficient infinite-horizon reasoning via reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2026.

Dongjie Yang, Xiaodong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. Pyramidinfer: Pyramid kv cache compression for high-throughput llm inference. In *Findings of ACL*, 2024.

Zhicheng Yang, Zhijiang Guo, Yinya Huang, Yongxin Wang, Wenlei Shi, Yiwei Wang, Xiaodan Liang, and Jing Tang. Accordion-Thinking: Self-regulated step summaries for efficient and readable LLM reasoning. *arXiv preprint arXiv:2602.03249*, 2026.

Xubing Ye, Yukang Gan, Xiaoke Huang, Yixiao Ge, and Yansong Tang. Voco-llama: Towards vision compression with large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 29836–29846, 2025.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025. ACL 2025, Best Paper Award.

Haoyue Zhang, Hualei Zhang, Xiaosong Ma, Jie Zhang, and Song Guo. Lazyeviction: Lagged kv eviction with attention pattern observation for efficient long reasoning. In *Proceedings of ACL*, 2026.

Jintian Zhang, Yuqi Zhu, Mengshu Sun, Yujie Luo, Shuofei Qiao, Lun Du, Da Zheng, Huajun Chen, and Ningyu Zhang. Lightthinker: Thinking step-by-step compression. In *Proceedings of EMNLP*, 2025.

Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. Long context compression with activation beacon. *arXiv preprint arXiv:2401.03462*, 2024. ICLR 2025.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2023.

A Related work

We sort prior work by what it compresses and whether the compression is learned. Every method that learns a compressed form of the model's own output was validated on generations one to two orders of magnitude shorter than ours, and every method validated at our length selects among existing rows rather than learning new ones.

Learned compression of the model's own output. MEMENTO (Kontonis et al., 2026) trains 8B–32B models to segment their reasoning into blocks, write a textual summary of each block and continue from the summaries alone, with about $2.5\times$ lower peak KV; like the other text-summary methods of the introduction, its summary is text the model writes, whereas ours is a set of learned KV rows. Its models lose accuracy against their full-context counterparts on competition mathematics before reinforcement learning, whereas our recipe matches its full-KV counterpart with supervised training alone. LightThinker (Zhang et al., 2025) is the nearest learned-row method: at each thought boundary the model writes the thought into 9 cache tokens and drops the thought's raw KV, with outputs of at most 10K tokens. HybridThinker (Liu et al., 2026) keeps the last few steps raw, at about 2.5K-token outputs, and Breadcrumbs Reasoning (Monea et al., 2025) keeps one beacon token every 2–32 generated tokens, on synthetic tasks with at most 1,000 generated tokens. Dynamic Memory Compression (Nawrot et al., 2024) learns per head and step whether to append a KV pair or merge it into the last slot; it merges raw rows rather than writing new ones. Latent-reasoning methods replace the chain itself with vectors (Hao et al., 2024; Cheng & Van Durme, 2024; Tan et al., 2025; Kuzina et al., 2026), give up the readable trace, and are evaluated at GSM8K scale. None of these reports accuracy against a full-KV model on generations of our length.

Compression tokens for inputs, and where their rows sit. Our code rows are an instance of a device introduced for inputs: place a few learned tokens after a span, hide the span from everything that follows, and train end to end so its information must route through those tokens. Gist tokens (Mu et al., 2023), ICAE (Ge et al., 2024b), AutoCompressors (Chevalier et al., 2023), Activation Beacon (Zhang et al., 2024) and VoCo-LLaMA (Ye et al., 2025) all compress a fixed input once, before generation, and are evaluated with short outputs; we compress the generated stream tens of times inside one response. None of them places its rows inside the span they replace: gist tokens are ordinary tokens, accumulated Activation Beacons occupy consecutive positions in the compacted stream, and LightThinker writes its rows at the thought boundary. The closest methods reuse an existing token's position (Pang et al., 2024; Qin et al., 2024) or defer position to retrieval time (Mohtashami & Jaggi, 2023). Our appended-position ablation (Section 4.4) is therefore our own model with this prior convention swapped in, not a reproduction of any one method. The nearest published configuration, LightThinker's token-level variant (every 6 generated tokens into 2 cache tokens at appended positions), is reported by its authors 5–6 points below their thought-level scheme on GSM8K-class tasks.

Eviction and prompt compression. Generic evictors keep recent tokens and heavy hitters by accumulated attention (Zhang et al., 2023; Liu et al., 2023; Oren et al., 2024; Xiao et al., 2024b; Ge et al., 2024a; Adnan et al., 2024), and reasoning-specific methods score tokens by importance and redundancy (Cai et al., 2025), compress from a window of recent queries (Song et al., 2025), or evict on Qwen3-4B with outputs of up to 16K tokens (Zhang et al., 2026; Chang et al., 2026); random eviction within the reasoning phase has been found to match the scored methods (Wang et al., 2026). Prompt compressors shrink a long input once after prefill (Li et al., 2024; Cai et al., 2024; Feng et al., 2025; Fu et al., 2025; Kim et al., 2025) and do not touch the generated stream. All of these choose among rows that exist; our model writes new rows, trained for the queries that will read them.

Allocating the budget across layers and heads. Heterogeneous budgets have been set per head from attention statistics (Ge et al., 2024a; Feng et al., 2025) or retrieval-head probes (Fu et al., 2025; Tang et al., 2025), and per layer from fixed pyramids (Cai et al., 2024; Yang et al., 2024) or measured statistics (Wang et al., 2024; Wan et al., 2025; Qin et al., 2025). Reported gains over uniform are modest outside very small budgets, and no decode-time reasoning compressor ablates the allocation. We measure each layer's value on the trained model itself, by cutting its rows (Section 5.1).

Complementary reductions. Sparse-attention methods keep every row and reduce what each step reads (Tang et al., 2024; Xiao et al., 2024a; Yuan et al., 2025; Gao et al., 2025); quantization (Liu et al., 2024; Hooper et al., 2024), latent KV (DeepSeek-AI, 2024) and sharing across heads or layers (Ainslie et al., 2023; Brandon et al., 2024) shrink each row; and shorter chains of thought (Xia et al., 2025; Xu et al., 2025) reduce the number of generated tokens. Each composes with a reduction in the number of rows kept.

B EXPERIMENTAL SETUP

B.1 TRAINING

Table 2: Training and evaluation setup shared by every arm.

Base model	Qwen3-4B-Base
SFT data	OpenMathReasoning-derived math SFT, 400K prompt/response pairs, fixed order
Response tokens per arm	2B (about 936 steps at effective batch 256)
Block size C / code slots $K_{\max}$ / mean quota	512 / 256 / 128
Optimizer, schedule	AdamW, lr 1e-4 WSD
Generation cap at eval	32768 tokens
Hardware	one node per run, either 4× NVIDIA GB300 or 8× NVIDIA H100-80GB; FlexAttention, bf16 autocast. A fixed data order sets each step's examples independently of the number of GPUs, so an arm is the same run on either machine

The learning rate warms up linearly over the first 3% of the 2B response tokens, holds at 10^{-4} until 80% of the budget, and decays linearly to 10^{-5} over the last 20%; AdamW with $\beta = (0.9, 0.95)$, weight decay 0.1 and gradient clipping at 1.0; effective batch 256 sequences (about 2.1M response tokens per step), 936 steps. The held-out set is the first 192 pairs.

Seed-to-seed noise. The data seed sets the random stream that draws each training step's examples from the 400K-pair pool, so two seeds train on the same pool and the same token budget but on different draws in a different order. Retraining the full-KV model on a different data seed shifts the suite score by 0.8 points (1 s.d. of the paired difference), the grey band of the figures. Two additions to the loss were tried under the same protocol and did not help: keeping the last 16 tokens of each block uncompressed (41.6 against 41.5 for the uniform setting), and distillation from the frozen full-KV model (39.9).

B.2 COMPETITION SUITE AND PROTOCOL

Construction. The set list was fixed before any arm was evaluated on it: AIME 2022–2026, HMMT February 2023–2025, BRUMO, CMIMC and SMT 2025 from MathArena, and AMC 12 2022–2023 from the AI-MO validation sets (Table 3). Two further sets, APEX 2025 and BeyondAIME, were left out before evaluation because a 4B model fine-tuned on our data solves too few of their problems to separate arms. Every arm is scored on the same problems with the same samples, cap and grader, so every comparison in the paper is a paired difference on identical problems.

Why pool. On our paired data a single 30-problem set puts the 95% interval on a gap between two arms at roughly ± 5 points: a full-KV replicate differing only in its data seed comes out significantly worse than the original on AIME 2024 and better on AIME 2025. Pooling 446 problems brings the half-width to 1.5 points. Cheaper metrics do not substitute for it: across a $4\times$ range of budget, held-out CE moves by 0.010 while the suite moves by 4.0 points (Appendix D.3).

Table 1 protocol. All SFT arms fine-tune Qwen3-4B-Base on the same 2B response tokens in the same order, with $C{=}512$. KV/512 counts persistent KV rows per 512 generated tokens per KV head; for R-KV, which keeps a constant number of rows, it is the measured residency over its generations. MATH-500 is avg@4 with the strict last-boxed grader. The three simple eviction policies were run

Table 3: The 446-problem competition suite by competition: source, release period, number of problems, problems that also occur in the SFT prompts (word 13-gram overlap $\geq$ 50% with a single training prompt), and avg@8 at the 32K cap for each arm. Arms: full KV (vanilla SFT), ABRIDGEKV per-layer, uniform allocation, appended code positions, LightThinker (thought-level, our stack) and R-KV at budget 4096 on the vanilla weights. A single set's 95% interval on a paired gap is about ± 5 points (Appendix B.2), so per-set differences are not individually meaningful.

Set	source	released	problems	in SFT	full KV	per-layer	uniform	appended	LightThinker	R-KV 4096
AIME 2022	AI-MO/aimo-validation-aime (url 2022)	pre-2025	30	19	42.1	41.7	42.1	33.8	25.8	40.4
AIME 2023	MathArena/aime_2023_I+II	pre-2025	30	17	40.0	40.4	35.8	31.7	21.7	36.2
AIME 2024	MathArena/aime_2024_I+II	pre-2025	30	–	50.8	50.0	42.9	34.2	36.7	52.9
AIME 2025	MathArena/aime_2025	2025+	30	–	37.1	33.8	34.2	27.5	25.0	30.8
AIME 2026	MathArena/aime_2026	2025+	30	–	36.2	38.3	37.1	24.2	29.2	33.8
AMC 12 2022–2023	AI-MO/aimo-validation-amc	pre-2025	83	32	75.6	75.2	73.2	67.5	66.1	76.7
HMMT Feb 2023	MathArena/hmmt_feb_2023	pre-2025	30	–	32.5	32.1	29.2	22.9	25.4	32.9
HMMT Feb 2024	MathArena/hmmt_feb_2024	pre-2025	30	–	27.1	24.6	21.7	20.4	18.8	26.2
HMMT Feb 2025	MathArena/hmmt_feb_2025	2025+	30	–	22.5	22.9	24.6	15.4	16.2	22.1
BRUMO 2025	MathArena/brumo_2025	2025+	30	1	50.4	45.0	44.2	41.7	36.7	44.2
CMIMC 2025	MathArena/cmimc_2025	2025+	40	–	19.1	21.6	20.9	16.2	13.8	18.8
SMT 2025	MathArena/smt_2025	2025+	53	–	44.1	42.2	42.2	36.8	32.8	43.4

in the HuggingFace decoder on a 60-problem subset (AIME 2025 and HMMT Feb 2025), paired against full KV on those problems († in the table).

Overlap with the training data. A problem is flagged when at least half of its word 13-grams occur in a single SFT training prompt. 69 of the 446 problems are flagged, all from AIME 2022–2023 and AMC except one borderline BRUMO problem; none from AIME 2024–2026, HMMT, CMIMC or SMT. Overlap raises absolute accuracy for every arm (full KV 69.2 on flagged problems, 39.2 on the rest), but all arms share the training data, so paired differences remain like for like. Table 4 repeats the analysis on subsets. Every difference to full KV keeps its sign on every subset, and the significant ones stay significant without the overlapping problems. The per-layer-versus-uniform difference is positive on every subset ($+1.6$ overall, $+1.1$ without overlap, $+2.9$ before 2025, $+0.1$ on 2025+ problems); each subset holds about half the suite, so its interval spans about four points and only the pooled suite resolves a difference of this size.

Table 4: Paired differences on subsets of the suite (avg@8 points, 95% bootstrap interval over problems). The first five columns are differences to full KV on the same problems; the last is per-layer minus uniform. "Overlapping" problems are the 69 that occur in the SFT prompts (Table 3).

			difference to full KV					
Subset	problems	full KV	per-layer	uniform	appended	LightThinker	R-KV 4096	per-layer − uniform
All 446 problems	446	43.8	-0.8 [$-2.3, +0.8$]	-2.3 [$-3.8, -0.9$]	-8.5 [$-10.3, -6.8$]	-10.5 [$-12.5, -8.7$]	-1.2 [$-2.7, +0.3$]	$+1.6$ [$+0.1, +3.0$]
No overlap with the SFT data	377	39.2	-0.8 [$-2.4, +0.9$]	-1.9 [$-3.5, -0.3$]	-7.8 [$-9.6, -5.9$]	-9.2 [$-11.3, -7.2$]	-1.0 [$-2.6, +0.6$]	$+1.1$ [$-0.4, +2.8$]
Released 2025 or later	213	35.2	-0.9 [$-3.3, +1.3$]	-1.0 [$-3.2, +1.1$]	-7.6 [$-10.2, -5.2$]	-9.3 [$-12.3, -6.6$]	-2.4 [$-4.8, -0.3$]	$+0.1$ [$-1.9, +2.3$]
Released before 2025	233	51.7	-0.6 [$-2.7, +1.3$]	-3.5 [$-5.5, -1.5$]	-9.3 [$-11.7, -6.9$]	-11.6 [$-14.2, -8.9$]	-0.1 [$-2.1, +2.0$]	$+2.9$ [$+0.9, +5.0$]
Overlapping problems only	69	69.2	-0.7 [$-4.5, +2.9$]	-4.7 [$-8.2, -1.3$]	-12.5 [$-17.2, -7.8$]	-18.1 [$-23.6, -13.0$]	-2.5 [$-6.7, +1.4$]	$+4.0$ [$+0.5, +7.4$]

C BASELINES

C.1 R-KV

R-KV (Cai et al., 2025) scores each generated token by attention importance minus key redundancy and keeps the highest-scoring rows up to a fixed budget. We port it to the vLLM version we serve with and run it on the full-KV checkpoint with the same problems, sampling and grader (Table 5).

Budget matching. R-KV's budget is a constant number of rows; ours is a rate. We match them by measured residency: over the suite's samples a budget of 4096 leaves about 12% more generated rows resident than our 4:1 arm, and 2048 about 12% more than our 8:1 arm, so each R-KV setting has slightly more cache than the arm it is compared with.

Served form. R-KV's reference code selects tokens per KV head, which a paged cache cannot honour, since dropping a token frees it for every head; we follow R-KV's own SGLang and vLLM ports, which reduce the score to one decision per token. We enable its opt-in flag that returns

Table 5: R-KV against the arm it is paired with at each budget, on the same 446 problems, the same sampling and the same grader. KV/512 = persistent rows per 512 generated tokens per KV head; R-KV's is its *measured* mean residency, so at both settings it holds about 12% more cache than the arm beside it. trunc. = % of samples that reach the 32K cap. Δ is the paired per-problem difference against that partner arm, bootstrapped on the 446 common problems. Accuracy separates them only at the tighter budget; truncation and generated length separate them at both.

Arm	KV/512	Suite avg@8	Δ vs. partner	trunc. %	gen. tokens
vanilla SFT (full KV)	512	43.8 [39.8, 47.6]	–	3.4	13,162
ABRIDGEKV, per-layer 4:1	128	43.0 [39.1, 46.9]	–	3.2	13,257
R-KV, budget 4096	144	42.6 [38.5, 46.6]	-0.4 [-2.1, +1.2]	31.7	18,428
ABRIDGEKV, uniform 8:1	64	39.4 [35.5, 43.2]	–	3.6	13,673
R-KV, budget 2048	74	35.6 [31.7, 39.5]	-3.8 [-5.5, -2.2]	55.8	22,230

evicted pages to the allocator, without which eviction saves no memory. Our port reproduces the released scoring bit for bit on every configuration we tested, uses the hyperparameters of R-KV's own evaluation, and compresses every 64 decoding steps rather than 128, which keeps the cache closer to the budget at a higher cost per token. Every request in both reported runs was compacted.

C.2 LIGHTTHINKER

LightThinker (Zhang et al., 2025) compresses a model's own reasoning online into learned KV entries. We compare against its mechanism trained on our data and served in our engine rather than against its released checkpoints, which differ in base model, data and decoding.

The arm. A thought ends at a paragraph break; the model then writes 9 cache tokens at appended positions under LightThinker's attention mask and continues from them, and the thought's raw KV is dropped at inference. Data, recipe and the 2B-token budget are those of every other arm; the served model matches the training-time forward pass (argmax agreement 0.996). Thoughts end every 53 tokens on average, so the arm keeps 97 rows per 512 generated tokens, between our 8:1 and 4:1 arms.

Result. It scores 33.3 on the suite, $-9.8\,[-11.5, -8.0]$ paired against ABRIDGEKV, and 2.0 points below our appended-position ablation, which shares its positions but compresses 512-token blocks. The compression unit explains most of the gap: at a fixed ratio, the block size sets how many lossy compressions a late query reads through and how much recent text stays exact (Table 6). It also writes longer, 14.9K tokens on average against 13.2K for full KV (Figure 4a). Its time to finish is an upper bound: its compression is triggered by a sampled token, so it ran with synchronous scheduling while every other arm ran asynchronously.

Table 6: Compression events a late query reads through, for our mean generation (13.2K tokens) and at the 32K cap, and the exact window (the open block, $C/2$ tokens in expectation).

block C	events (mean gen.)	events (at cap)	exact window
1024	13	31	512
512 (ours)	26	62	256
256	51	124	128
$\approx$53 (LightThinker, measured)	249	618	26

D ADDITIONAL RESULTS

D.1 PER-LAYER VERSUS PER-HEAD ALLOCATION

Figure 7 repeats the truncation measurement of Section 2.3 head by head on the per-layer model. In the layers cut to 64 rows, doubling any single head gains nothing; in the layers at 128, no head is expensive; in the six layers raised to 256, the value sits in one or two heads. So the per-layer

Table 7: Budget allocation at a fixed mean of 128 code rows per block (all other settings identical). Held-out CE on 192 held-out SFT pairs; MATH-500 avg@4; suite avg@8 at 32K as in Table 1. The per-head row lets a head keep up to all 512 rows, with a floor of 32, nested inside the per-layer schedule; paired against the per-layer headline it is -0.6 $[-2.1, +0.9]$. The last row sets per-head quotas from how concentrated each head's attention is in the full-KV model, which raises the early layers; paired against the per-layer model it is -1.9 $[-3.3, -0.5]$.

Schedule	held-out CE	MATH-500	Suite avg@8 [95% CI]
uniform (every head 128)	0.4227	88.3	41.5 [37.6, 45.3]
per layer, measured on the trained model (ABRIDGEKV)	0.4215	87.5	43.0 [39.1, 46.9]
per head, measured, $K_{\max}$=512 (nested)	0.4209	–	42.4 [38.5, 46.3]
per head, from full-KV attention statistics	0.4231	–	41.1 [37.2, 45.0]

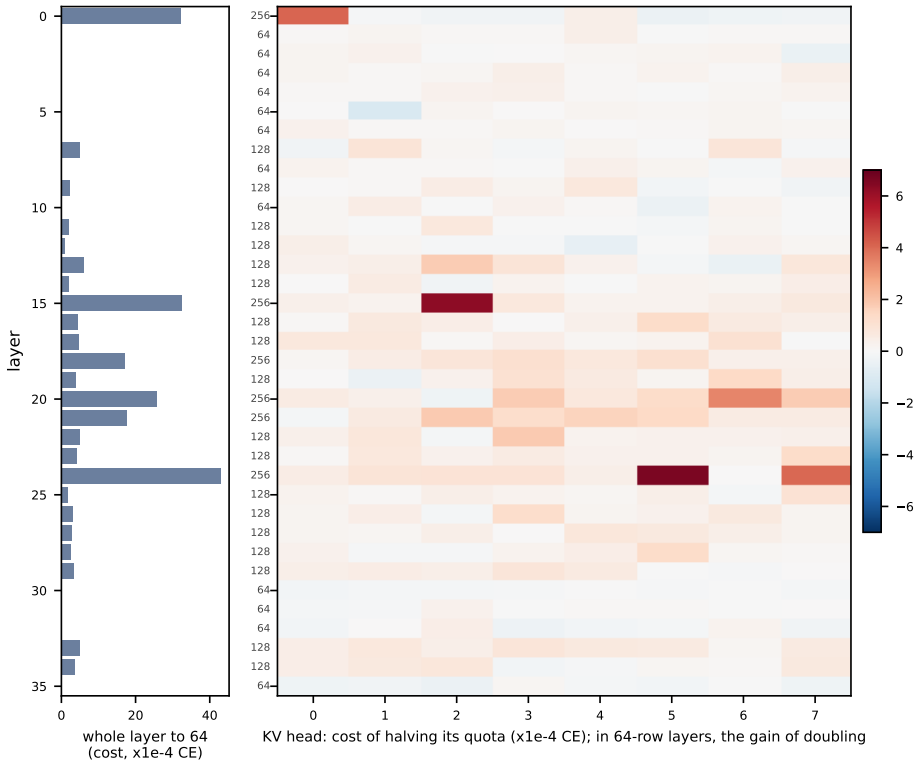


Figure 7: Where a layer's code rows are worth something, head by head. On the trained per-layer model, each (layer, KV head) cell shows the held-out CE cost of halving that one head's quota (in 64-row layers, the gain of doubling it), on the full 192-pair held-out set; the bars at left repeat the whole-layer cost of Figure 6. Noise floor: 0.4×10^{-4} (s.d. of head cells in layers whose whole-layer cost is nil).

schedule already captures nearly all of the value that can be moved at this mean, and a per-head schedule trained from scratch (Table 7) is within noise of it. It would also cost to serve: a paged engine keeps one row count per layer, so ragged per-head quotas either pad every head to the layer's largest ($1.53\times$ the memory of the per-layer schedule) or need a packed side cache, which adds about 29 ms of fixed overhead per decode step. We therefore allocate per layer.

D.2 WHAT THE CODE ROWS CARRY

Figure 8 adds two views to Section 5.2. Swapping block b's code rows in only the 12 layers cut to 64 rows leaves next-block recall at 0.99, and keeping only those layers intact is as bad as swapping every

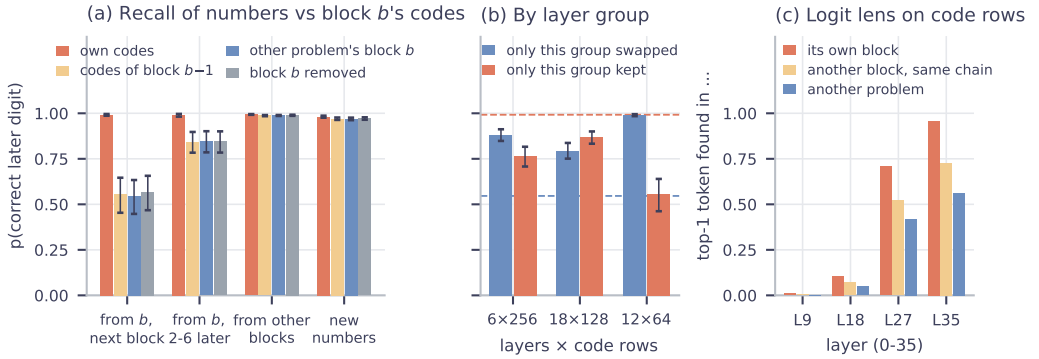


Figure 8: What the code rows carry (115 AIME chains, 895 intervened blocks). (a) Probability of a number's later digits, by where it was first written, with block b's code rows intact, replaced, or removed (95% intervals over chains). (b) Numbers from b reused in the next block, with one layer group swapped or only one kept; dashed: all intact, all swapped. (c) How often a code row's logit-lens top token occurs in its own block, another block, or another problem.

layer (0.55); the 256- and 128-row layers carry the content (kept alone, 0.77 and 0.87). Projected onto the vocabulary after layer 27, a code row's most likely token occurs in its own block for 71% of blocks, against 52% for another block of the same chain and 42% for another problem's.

D.3 Block size, code budget and training cost

Within one block size, halving the code budget costs far more on the suite than held-out CE suggests. At C=512, going from K=128 (4:1) to K=64 (8:1) moves held-out CE by 0.005 but the pooled suite by 2.1 points, 41.5 to 39.4. Doubling the budget the other way, K=256 (2:1) moves held-out CE by a further 0.004 and the suite by 1.9 points to 43.4, which is -0.4 $[-1.8, +1.0]$ against full KV: lossless within the interval, at half the compression. The remaining rows of Table 8 vary the block size at a fixed ratio (Figure 5); held-out CE is not comparable across block sizes, since the held-out set is defined by blocks per example.

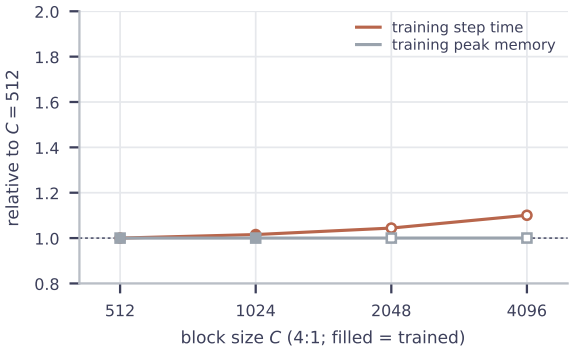


Figure 9: Training cost of the block size at 4:1, relative to C=512: step time and peak memory for one forward and backward pass over a 16K-token sequence on one H100 (filled: block sizes we trained). C=256, not shown, takes 1.9× as long, from partly filled attention tiles and twice as many blocks.

Larger blocks cost little more to train (Figure 9). With activation checkpointing, peak memory is set by the sequence length and is the same at every block size; step time rises by 2% at C=1024 and 10% at C=4096, from the attention inside each open block.

Table 8: Block size and code budget sweep (uniform quotas, bitrev positions, $K_{\max}$=256, same 2B tokens). Held-out CE is comparable only within one block size (the held-out set is defined by blocks per example). MATH-500 avg@4; suite avg@8 at 32K as in Table 1.

C	mean K	ratio	held-out CE	MATH-500	Suite avg@8 [95% CI]
512	128	4:1	0.4227	88.3	41.5 [37.6, 45.3]
512	64	8:1	0.4281	86.7	39.4 [35.5, 43.2]
512	256	2:1	0.4184	87.6	43.4 [39.4, 47.2]
256	64	4:1	0.4246	86.9	40.7 [36.7, 44.5]
1024	256	4:1	0.4260	87.2	42.5 [38.6, 46.3]

D.4 Coding domain at prototyping scale

Every number in the main text is on mathematical reasoning. To test whether the recipe carries to another long-generation domain we repeated the comparison on code at a reduced scale: 600M response tokens per run (0.3 of the main budget), with held-out cross-entropy as the metric. The data are 120K problem–solution pairs from OpenCodeReasoning (Ahmad et al., 2025), each solution a DeepSeek-R1 reasoning trace of 512–16K tokens ending in a Python program (median 7.0K tokens). At the same scale on mathematics, two uniform models that differ only in an initially inactive projection differ by 0.0005, which sets the resolution of the comparison. Table 9 shows the same

Table 9: Coding domain at prototyping scale: held-out cross-entropy (nats per response token) after 600M response tokens on OpenCodeReasoning, and the difference to the full-KV model trained identically. All compressed arms keep 128 code rows per 512-token block (4:1). Resolution of the comparison: 0.0005 (a null pair at the same scale on mathematics).

Arm	Held-out CE	vs. full KV
Full KV	0.5450	–
Uniform, 128 rows	0.5558	+0.0108
Per-layer, schedule measured on mathematics	0.5544	+0.0094
Per-layer, schedule measured on code	0.5544	+0.0094

ordering as on mathematics: full KV, then per-layer, then uniform. Compression at 4:1 costs 0.011 nats on code (2% of the full-KV cross-entropy), and the per-layer schedule recovers 0.0014 of it. Re-measuring the schedule on the coding model gives the same layers to within one at each budget level (five of the six 256-row layers and eleven of the twelve 64-row layers), and the two schedules reach the same held-out cross-entropy: the value a layer places on its code rows is a property of the model rather than of the training domain. Accuracy and length on code remain to be measured, which needs full-budget training and an execution-based benchmark.

E Serving and correctness

Implementation. The patch adds compressed decoding to vLLM's v1 engine. When a request completes a block of C tokens, the scheduler appends its K pause tokens; their KV at every layer is written into the block's persistent slots, and the block's C raw rows are freed. The pause tokens run in the same forward pass as other requests' decode tokens, which removes a separate engine step; merged steps are captured as piecewise CUDA graphs, with the same capture range for every arm, baselines included. A per-layer schedule is served as one KV cache group per quota level, so a layer with quota q stores q rows per block and nothing else; uniform arms use a single group.

Time to finish. Each arm's time to finish is taken at the best concurrency cap of a doubling grid. AbridgeKV cannot run 256 sequences on one H100 (at 128 its cache is already 92% full), so its value is bounded by memory; the uniform setting, which does run at 256, is slower there than at 128, so we expect the further gain to be small.

Parity. Every accuracy in the paper is measured on the served model, so the serving path must compute the same function as the trainer (Table 10). The trainer's single forward pass under the

block mask and an incremental generator that physically deletes each completed block's KV agree on per-token cross-entropy to within 4.5×10^{-4}; a deliberately wrong schedule differs by two orders of magnitude. On every serving path the vLLM engine agrees with that generator on the top-1 token at 99.5% or more of positions across block boundaries. As a negative control, we replay 6,000-token vLLM generations through the same weights with full attention over the uncompressed history: on the tokens where that model and the compressing generator disagree by more than 0.05 nats, the served model follows the compressing generator on 287 of 297 tokens (uniform), 469 of 479 (per-layer, masked) and 294 of 311 (per-layer, physical cache groups), so the served models do not attend to compressed-away history.

Table 10: Train/serve parity checks.

Check	Setting	Result
Trainer vs. incremental generator (per-token CE)	vph checkpoint, 3 held-out sequences, trained schedule	mean $\lvert\Delta\rvert$ 3.5e-4 to 4.5e-4, max 1.3e-2 to 2.3e-2
Same, deliberately wrong (uniform) schedule	control	mean $\lvert\Delta\rvert$ 1.6e-2 to 2.2e-2, max 4.0
HF reference vs. vLLM packed KV (greedy tokens)	uniform128, hetero_proxy, vph; 510 prompt + 16 generated (one block boundary)	identical greedy tokens
Multi-boundary HF vs. vLLM parity	per-layer headline, physical cache groups; 510-token prompt, 1,200 teacher-forced tokens (two block boundaries)	top-1 agreement 0.995, mean $\lvert\Delta \log p\rvert$ 0.006